CpE 3110
Computer Organization & Design

Dr. Minsu Choi

Instructions: Language of the Computer

CHAPTER 2

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation (especially in hardware)
- Many modern computers also have simple instruction sets

The ARMv8 Instruction Set

- A subset, called LEGv8, used as the example throughout the book
- LEGv8 is intended as a pun on ARMv8.
- Commercialized by ARM Holdings (www.arm.com)
- Large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …
- Typical of many modern ISAs
  - See ARM Reference Data tear-out card (aka Green Card)

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
ADD a, b, c   // a gets b + c
```

- All arithmetic operations have this form
- *Design Principle 1:* Simplicity favors regularity
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Arithmetic Example

- C code:

```
f = (g + h) - (i + j);
```

- Compiled LEGv8 code:

```
ADD t0, g, h    # temp t0 = g + h
ADD t1, i, j    # temp t1 = i + j
SUB f, t0, t1   # f = t0 - t1
```

Register Operands

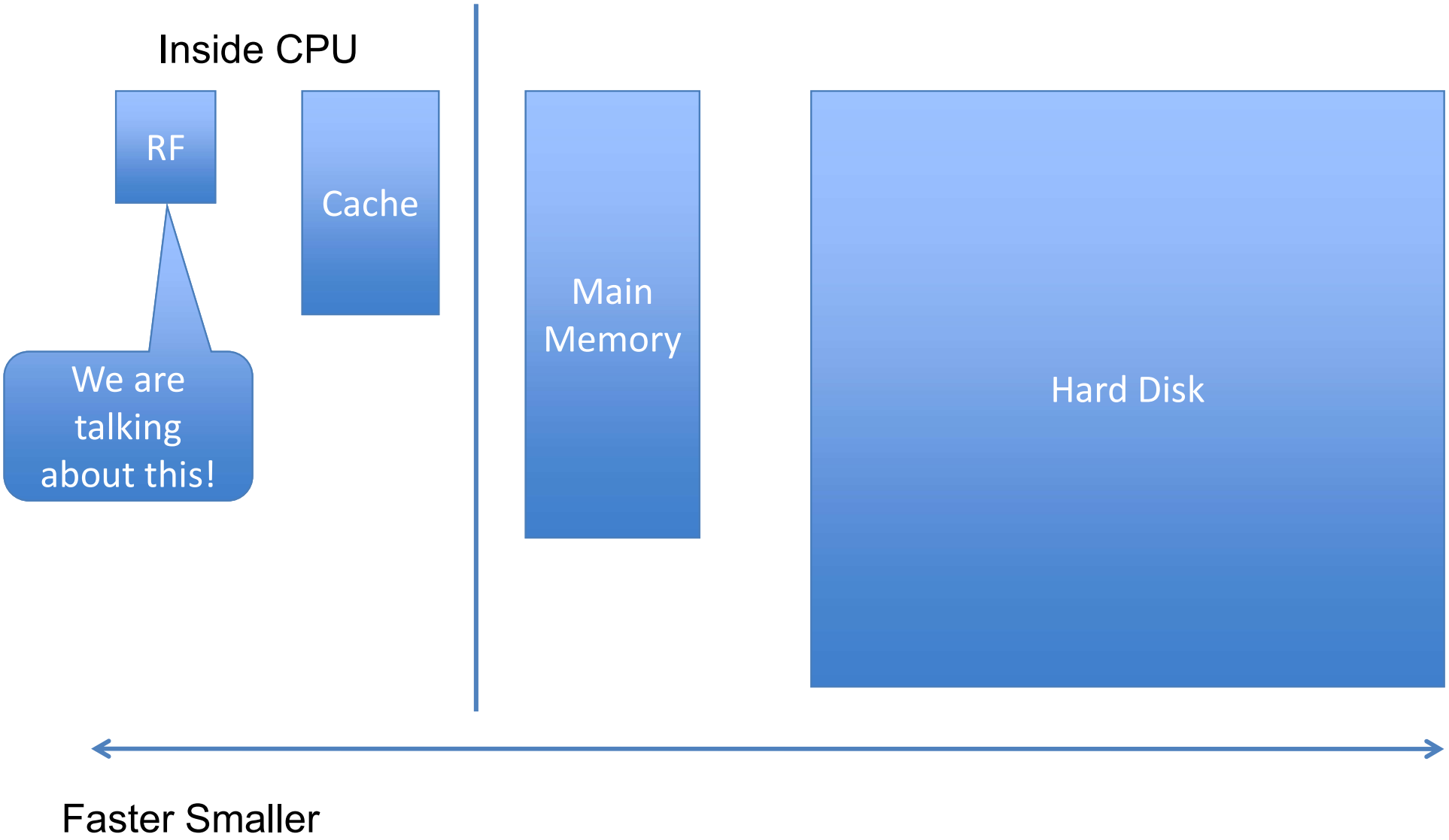
- Arithmetic instructions use register operands

- LEGv8 has a 32 × 64-bit register file (RF)
  - Use for frequently accessed data
  - 64-bit data is called a “doubleword”
    - 31 x 64-bit general purpose registers X0 to X30
  - 32-bit data called a “word”
    - 31 x 32-bit general purpose sub-registers W0 to W30

- *Design Principle 2:* Smaller is faster
  - c.f. main memory: millions of locations

- X0 – X7: procedure arguments/results
- X8: indirect result location register
- X9 – X15: temporaries
- X16 – X17 (IP0 – IP1): may be used by linker as a scratch register, other times as temporary register
- X18: platform register for platform independent code; otherwise a temporary register
- X19 – X27: saved
- X28 (SP): stack pointer
- X29 (FP): frame pointer
- X30 (LR): link register (return address)
- XZR (register 31): the constant value 0

Memory Hierarchy



Register Operand Example

- C code:

```
f = (g + h) - (i + j);
```

  - f, …, j in X19, X20, …, X23

- Compiled LEGv8 code:

```
ADD X9, X20, X21
ADD X10, X22, X23
SUB X19, X9, X10
```

Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - Load values from memory into registers
  - Store result from register to memory
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- LEGv8 does not require words to be aligned in memory, except for instructions and the stack

Memory Operand Example

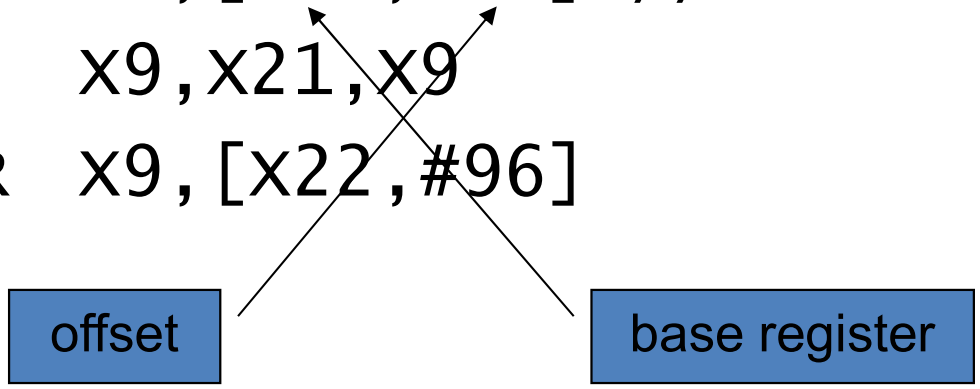
- C code:

```
A[12] = h + A[8];
```

  - h in X21, base address of A in X22

- Compiled LEGv8 code:
  - Index 8 requires offset of 64

```
LDUR  X9,[X22,#64] // U for "unscaled"
ADD   X9,X21,X9
STUR  X9,[X22,#96]
```



Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important for speed!

Immediate Operands

- Constant data specified in an instruction

  `ADDI X22, X22, #4`
- No separate memory read is needed
- *Design Principle 3:* Make the common case fast
  - Small constants are common
  - Immediate operand avoids a load instruction

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32-bit word
  - 0 to +4,294,967,295

Signed Integers in LEGv8

- 2's complement method is used
- Negation is done by bit-wise invert then add 1
- Add hardware can be used for subtraction
- A – B = A + (-B)
- Ex) 1 – 1 = 1 + (-1) = 0
  - 0x00000001 + 0xffffffff = 0x00000000 w/ carryout bit of 1
  - Carryout bit ignored (no overflow)

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
  - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
    $= -1 \times 2^{31} + 1 \times 2^{30} + \ldots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Using 32 bits
  - –2,147,483,648 to +2,147,483,647

2s-Complement Signed Integers

- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^{n-1})$ can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Most-negative: 1000 0000 … 0000
  - Most-positive: 0111 1111 … 1111

Signed Negation

- Complement and add 1
  - Complement means 1 → 0, 0 → 1

$$x + \bar{x} = 1111...111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
  - +2 = 0000 0000 … 0010_2
  - –2 = 1111 1111 … 1101_2 + 1
    = 1111 1111 … 1110_2

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110

- In LEGv8 instruction set
  - LDURSB: sign-extend loaded byte
  - LDURB: zero-extend loaded byte

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- LEGv8 instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, …
  - Regularity!

Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

- Example: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

LEGv8 R-format Instructions



- Instruction fields
  - opcode: operation code
  - Rm: the second register source operand
  - shamt: shift amount (000000 for now)
  - Rn: the first register source operand
  - Rd: the register destination

R-format Example



ADD X9,X20,X21

1112_{ten}	21_{ten}	0_{ten}	20_{ten}	9_{ten}
10001011000_{two}	10101_{two}	000000_{two}	10100_{two}	01001_{two}

$1000\ 1011\ 0001\ 0101\ 0000\ 0010\ 1000\ 1001_{two}$ =

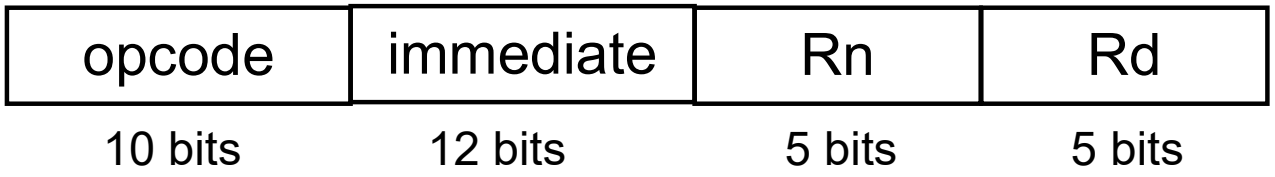
$8B150289_{16}$

LEGv8 D-format Instructions



- Load/store instructions
  - Rn: base register
  - address: constant offset from contents of base register (+/- 32 doublewords)
  - Rt: destination (load) or source (store) register number
  - Op2: for extensions. 00 for LEGv8.
- *Design Principle 3:* Good design demands good compromises
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

LEGv8 I-format Instructions



- Immediate instructions
  - Rn: source register
  - Rd: destination register
- Immediate field is zero-extended
- When decoding opcode, 11 bits are still decoded – making LSB as don't care (e.g., ADDI has opcode range 488 – 489 in Hex): for simple decoding logic and HW

Example

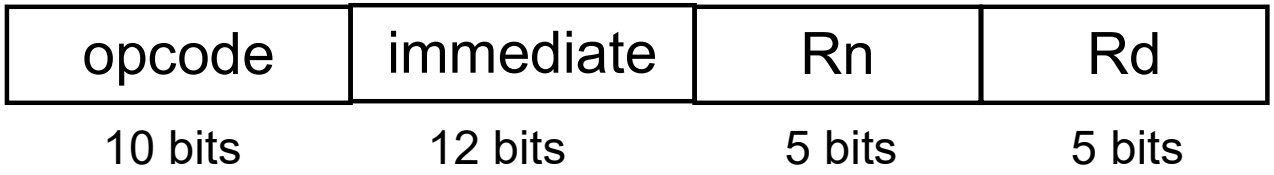
- LDUR X9, [X10, #240]
- ADDI X9, X9, #1
- STUR X9, [X10, #240]

LDUR X9, [X10, #240]



- D-format instruction
- Opcode: 7c2 in hex = 111 1100 0010 in 11-bit binary
- Offset: 240 in decimal = 0 1111 0000 in 9-bit signed binary integer
- Op2: 00 – reserved for future extensions, 00 for LEGv8
- Rn = 01010 (10) and Rt = 01001 (9)

ADDI X9, X9, #1



- I-format
- Opcode (I-format has 10-bit opcode. So, it occupies a range of 11-bit opcode): 488 – 489 in hex = 100 1000 100X in binary (X: don't care)
- Immediate: 1 in decimal = 0000 0000 0001 in 12-bit unsigned binary integer
- Rn and Rt: 9 in decimal = 01001 in binary

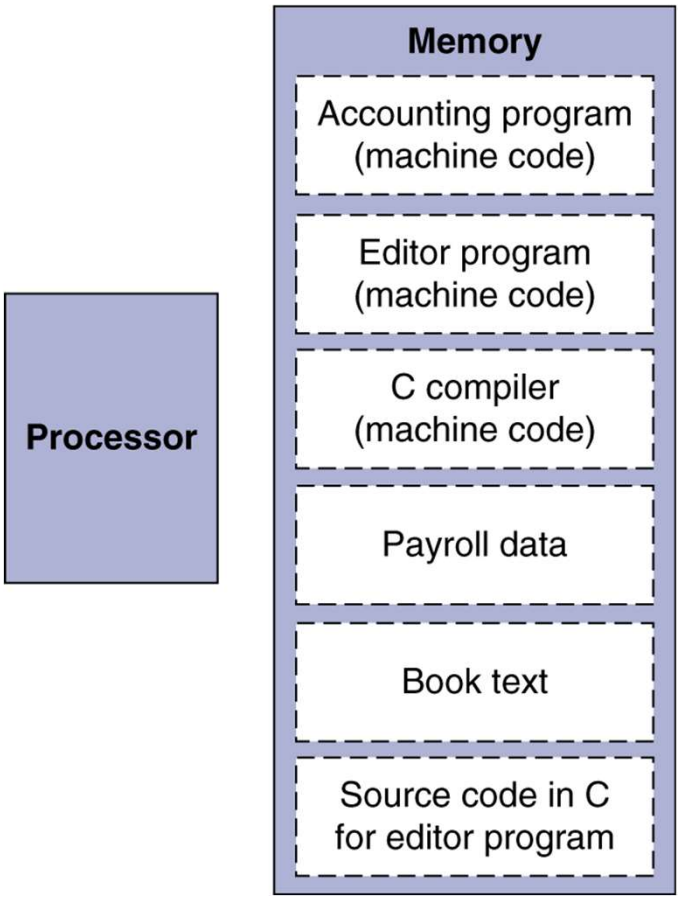
STUR X9, [X10, #240]



- D-format
- Opcode: 0x7c0 = $111\ 1100\ 0000_2$
- Offset: $240_{10} = 0\ 1111\ 0000_2$
- Op2: 00
- Rn: 01010
- Rt: 01001

Stored Program Concept

The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	LEGv8
Shift left	<<	<<	LSL
Shift right	>>	>>>	LSR
Bitwise AND	&	&	AND, ANDI
Bitwise OR	|	|	OR, ORI
Bitwise NOT	~	~	EOR, EORI

- Useful for extracting and inserting groups of bits in a word

Shift Operations

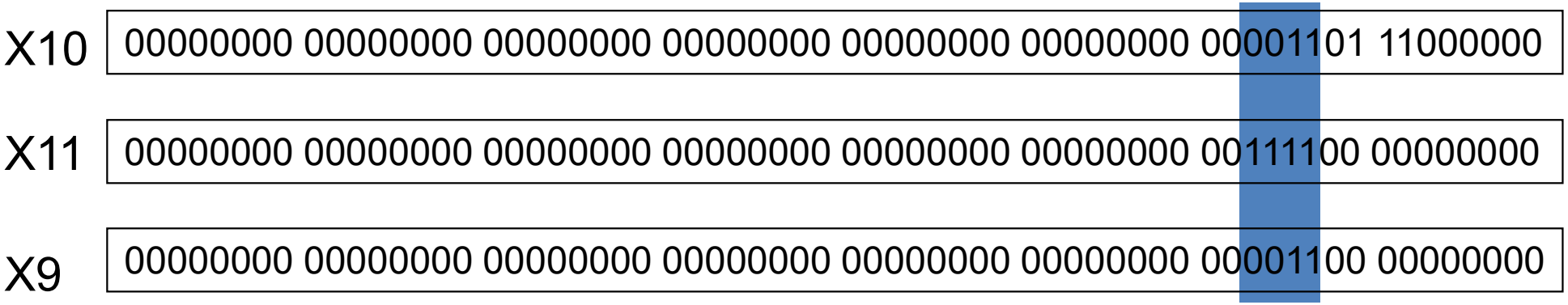


- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - LSL by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - LSR by *i* bits divides by 2^i (unsigned only)

AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

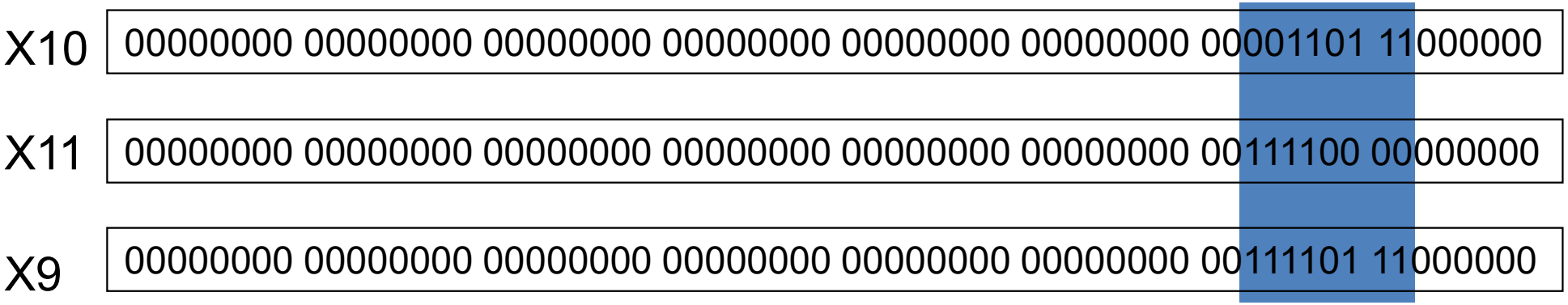
```
AND  X9,X10,X11
```



OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

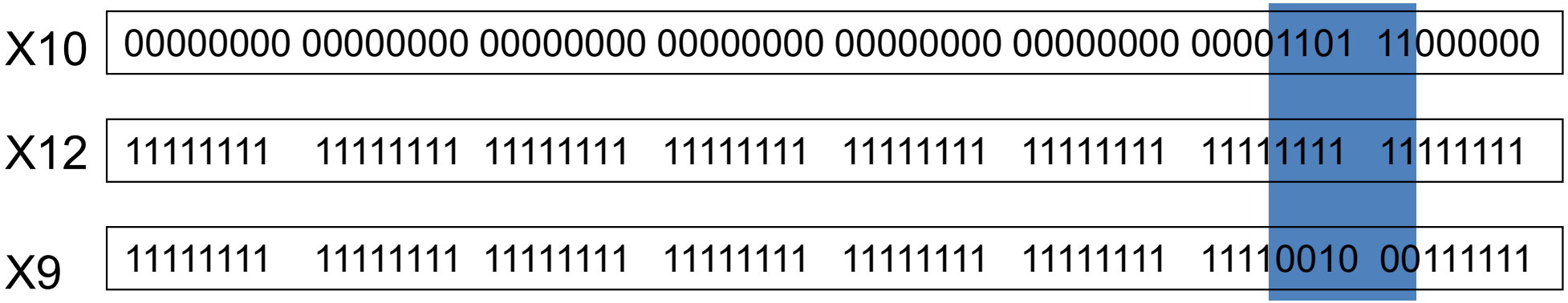
```
OR  X9,X10,X11
```



EOR Operations

- XOR operation
- Differencing operation
  - Set some bits to 1, leave others unchanged

```
EOR X9,X10,X12  // NOT operation
```



Conditional Operations

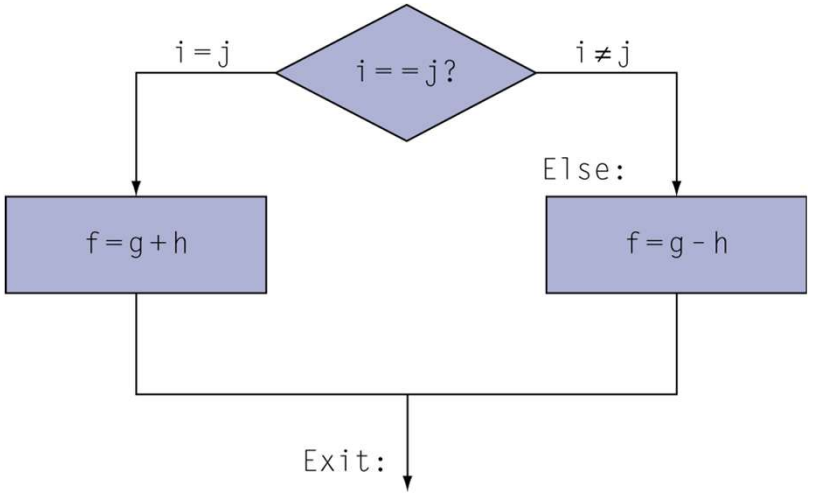
- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `CBZ register, L1`
  - if (register == 0) branch to instruction labeled L1;
- `CBNZ register, L1`
  - if (register != 0) branch to instruction labeled L1;
- `B L1`
  - branch unconditionally to instruction labeled L1;

Compiling If Statements

- C code:

```
if (i==j) f = g+h;
else f = g-h;
```

  - f, g, … in X19, X20, …



- Compiled LEGv8 code:

```
      SUB X9,X22,X23
      CBNZ X9,Else
      ADD X19,X20,X21
      B Exit
Else: SUB X9,X20,x21
Exit: …
```

Assembler calculates addresses

Compiling Loop Statements

- C code:

```
while (save[i] == k) i += 1;
```

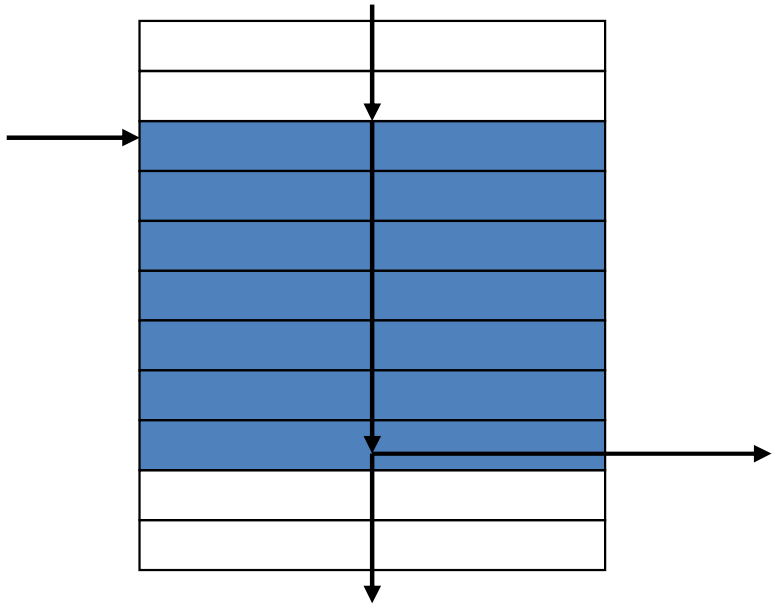
  - i in x22, k in x24, base address of save in x25

- Compiled LEGv8 code:

```
Loop: LSL  X10,X22,#3   // DW to B offset
      ADD    X10,X10,X25
      LDUR   X9,[X10,#0]
      SUB    X11,X9,X24
      CBNZ   X11,Exit
      ADDI   X22,X22,#1
      B      Loop
Exit: …
```

Basic Blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

More Conditional Operations

- Condition codes, set from arithmetic instructi on with S-suffix (ADDS, ADDIS, ANDS, ANDIS, S UBS, SUBIS)
  - negative (N): result had 1 in MSB
  - zero (Z): result was 0
  - overlow (V): result overflowed
  - carry (C): result had carryout from MSB

Continued,

- Use subtract to set flags, then conditionally branch:
  - **B.EQ** (equal)
  - **B.NE** (not equal)
  - **B.LT** (less than, signed), **B.LO** (less than, unsigned)
  - **B.LE** (less than or equal, signed), **B.LS** (less than or equal, unsigned)
  - **B.GT** (greater than, signed), **B.HI** (greater than, unsigned)
  - **B.GE** (greater than or equal, signed),
  - **B.HS** (greater than or equal, unsigned)

Conditional Example

- if (a > b) a += 1;
  - a in X22, b in X23

```
    SUBS X9,X22,X23  // use subtract to make comparison
    B.LE Exit        // conditional branch
    ADDI X22,X22,#1
Exit:
```

Signed vs. Unsigned

- Signed comparison
- Unsigned comparison
- Example
  - X22 = 1111 1111 1111 1111 1111 1111 1111 1111
  - X23 = 0000 0000 0000 0000 0000 0000 0000 0001
  - `X22 < X23 # signed`
    - –1 < +1
  - `X22 > X23 # unsigned`
    - +4,294,967,295 > +1

Branch Offset Calculation

```
Loop:   <inst>
        <inst>
        <inst>
        <inst>
        CBNZ X11, Exit
        <inst>
        B Loop
Exit:   <inst>
```

CBNZ X11, Exit

- CB-format: 8b opcode, 19b offset and 5b Rt
- Opcode (8b): 5a8 to 5af = 101 1010 1XXX-> tak ing the first 8 bits: 101 1010 1
- Offset (19b): 3 insts to skip to Exit -> 000 0000 0000 0000 0011
- Rt: 01011

B Loop

- B-format: 6b opcode and 26b offset
- Opcode: 0a0 to 0bf = 000 101X XXXX: taking the first 6b -> 000 101
- Offset: 6 insts to skip backwards -> -6
- We need to find 2's comp of +6 for -6.
- +6 = 00 0000 0000 0000 0000 0000 0110
- 1's comp: 11 1111 1111 1111 1111 1111 1001
- 2's comp: 11 1111 1111 1111 1111 1111 1010 = -6

Procedure Calling

- Steps required
  1. Place parameters in registers X0 to X7
  2. Transfer control to procedure
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register for caller
  6. Return to place of call (address in X30 LR link register)

Procedure Call Instructions

- Procedure call: branch with link

  `BL ProcedureLabel`

  – Address of following instruction (return address) put in X30

  – Jumps to target address

- Procedure return: branch to register

  `BR LR`

  – Copies LR (Link Reg – X30) to program counter

  – Can also be used for computed jumps

    - e.g., for case/switch statements

Leaf Procedure Example

- C code:

```
long long int leaf_example (long
long int g, long long int h, long
long int i, long long int j)
{ long long int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in X0, …, X3
  - f in X19 (hence, need to save X19 on stack)
  - long long int: 64-bit doubleword int data type

Leaf Procedure Example

- LEGv8 code:

```
leaf_example:
SUBI SP,SP,#24
STUR X10,[SP,#16]
STUR X9,[SP,#8]
STUR X19,[SP,#0]
ADD X9,X0,X1
ADD X10,X2,X3
SUB X19,X9,X10
ADD X0,X19,XZR
LDUR X10,[SP,#16]
LDUR X9,[SP,#8]
LDUR X19,[SP,#0]
ADDI SP,SP,#24
BR LR
```

Save X10, X9, X19 on stack
(see next page)

X9 = g + h

X10 = i + j

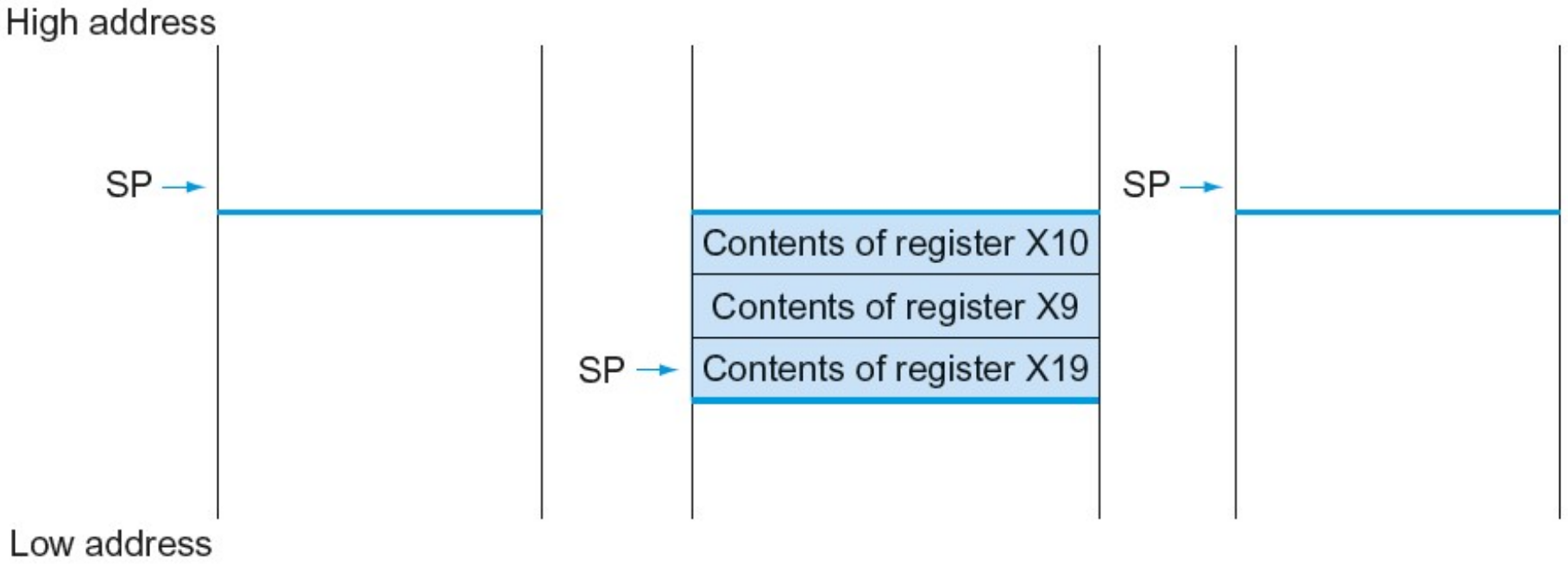
f = X9 – X10

copy f to return register

Resore X10, X9, X19 from stack

Return to caller

Local Data on the Stack



- Stack is used to save/restore previous values to prevent overwriting
- SP (Stack Pointer – X28) is used to keep track of the stack top

Register Usage

- X9 to X17: temporary registers
  - The callee does not need to preserve - optional
- X19 to X28: saved temp registers
  - If used, the callee saves and restores them using stack - mandatory

Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
long long int fact (long long int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

  - Argument n in X0
  - Result in X1

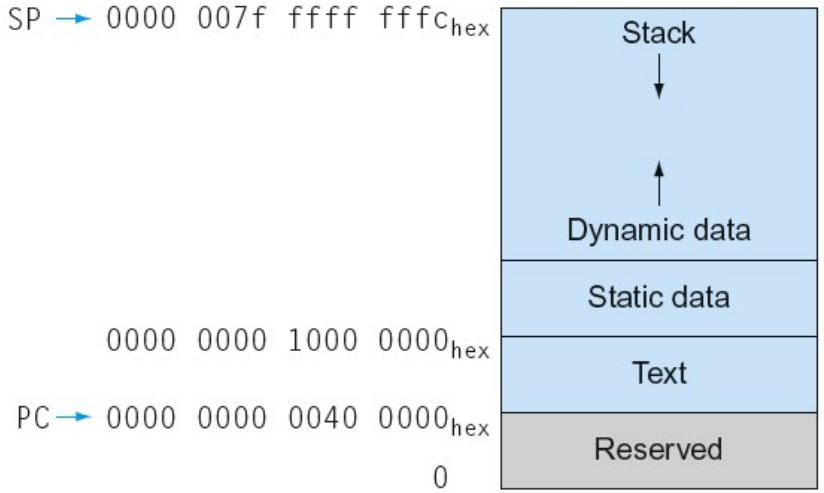
Non-Leaf Procedure Example

- LEGv8 code:

```
fact:
   SUBI SP,SP,#16      Save return address and n on stack
   STUR LR,[SP,#8]
   STUR X0,[SP,#0]
   SUBIS XZR,X0,#1     compare n and 1
   B.GE L1             if n >= 1, go to L1
   ADDI X1,XZR,#1      Else, set return value to 1
   ADDI SP,SP,#16      Pop stack, don't bother restoring values
   BR LR               Return
L1: SUBI X0,X0,#1      n = n - 1
   BL fact             call fact(n-1)
   LDUR X0,[SP,#0]     Restore caller's n
   LDUR LR,[SP,#8]     Restore caller's return address
   ADDI SP,SP,#16      Pop stack
   MUL X1,X0,X1        return n * fact(n-1)
   BR LR               return
```

Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword Operations

- LEGv8 byte/halfword load/store
  - Load byte:
    - `LDURB Rt, [Rn, offset]`
    - Sign extend to 64 bits in rt
  - Store byte:
    - `STURB Rt, [Rn, offset]`
    - Store just rightmost byte
  - Load halfword:
    - `LDURH Rt, [Rn, offset]`
    - Sign extend to 64 bits in rt
  - Store halfword:
    - `STURH Rt, [Rn, offset]`
    - Store just rightmost halfword

String Copy Example

- C code (naïve):
  - Null-terminated string

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

  - Addresses of x, y in X0, X1
  - i in X19

String Copy Example

- LEGv8 code:

```
strcpy:
    SUBI SP,SP,#8       // push X19
    STUR X19,[SP,#0]
    ADD X19,XZR,XZR     // i=0
L1: ADD X10,X19,X1      // X10 = addr of y[i]
    LDURB X11,[X10,#0]      // X11 = y[i]
    ADD X12,X19,X0      // X12 = addr of x[i]
    STURB X11,[X12,#0]      // x[i] = y[i]
    CBZ X11,L2          // if y[i] == 0 then exit
    ADDI X19,X19,#1     // i = i + 1
    B L1                // next iteration of loop
L2: LDUR X19,[SP,#0]    // restore saved X19
    ADDI SP,SP,#8       // pop 1 item from stack
    BR LR               // and return
```

32-bit or 64-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient
- For the occasional 32-bit constant

  `MOVZ: move wide with zeros`

  `MOVK: move wide with keep`

  `Both have 16b immediate field ( IW – wide immediate format)`
- Use with flexible operand (shift) – 0, 16, 32, 48

`MOVZ X9,#255,LSL 16`

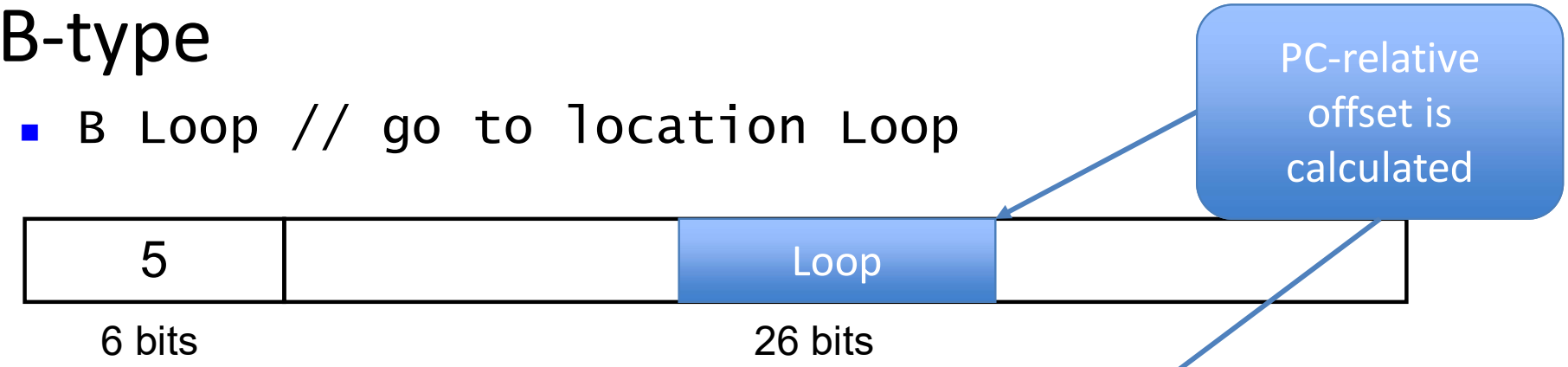
0000 0000 0000 0000	0000 0000 0000 0000	0000 0000 1111 1111	0000 0000 0000 0000

`MOVK X9,#255,LSL 0`

0000 0000 0000 0000	0000 0000 0000 0000	0000 0000 1111 1111	0000 0000 1111 1111

Branch Addressing

- B-type
  - `B Loop // go to location Loop`

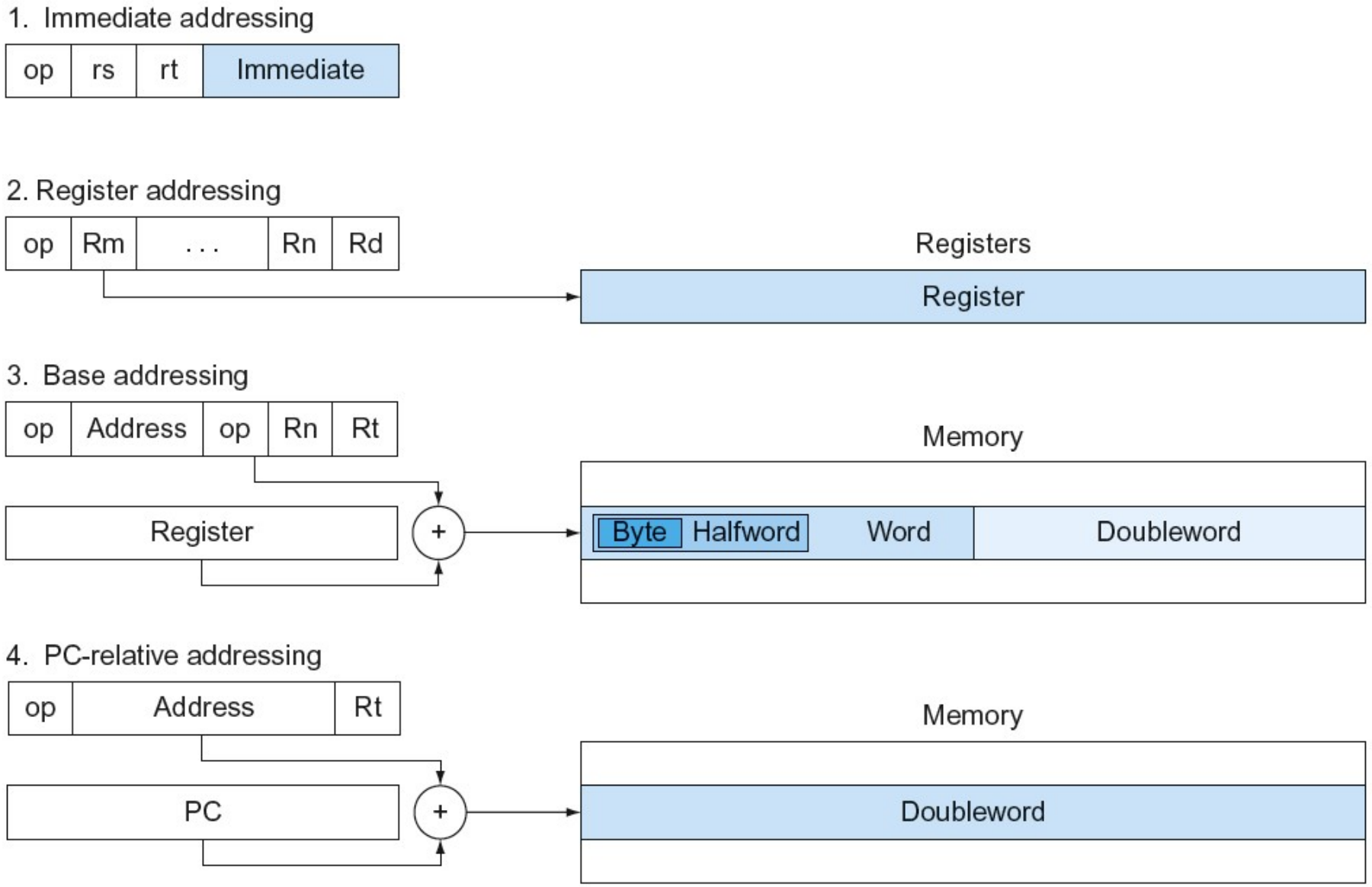


- CB-type
  - `CBNZ X19, Exit // go to Exit if X19 != 0`



- Both addresses are PC-relative
  - Address = PC + offset (from instruction)

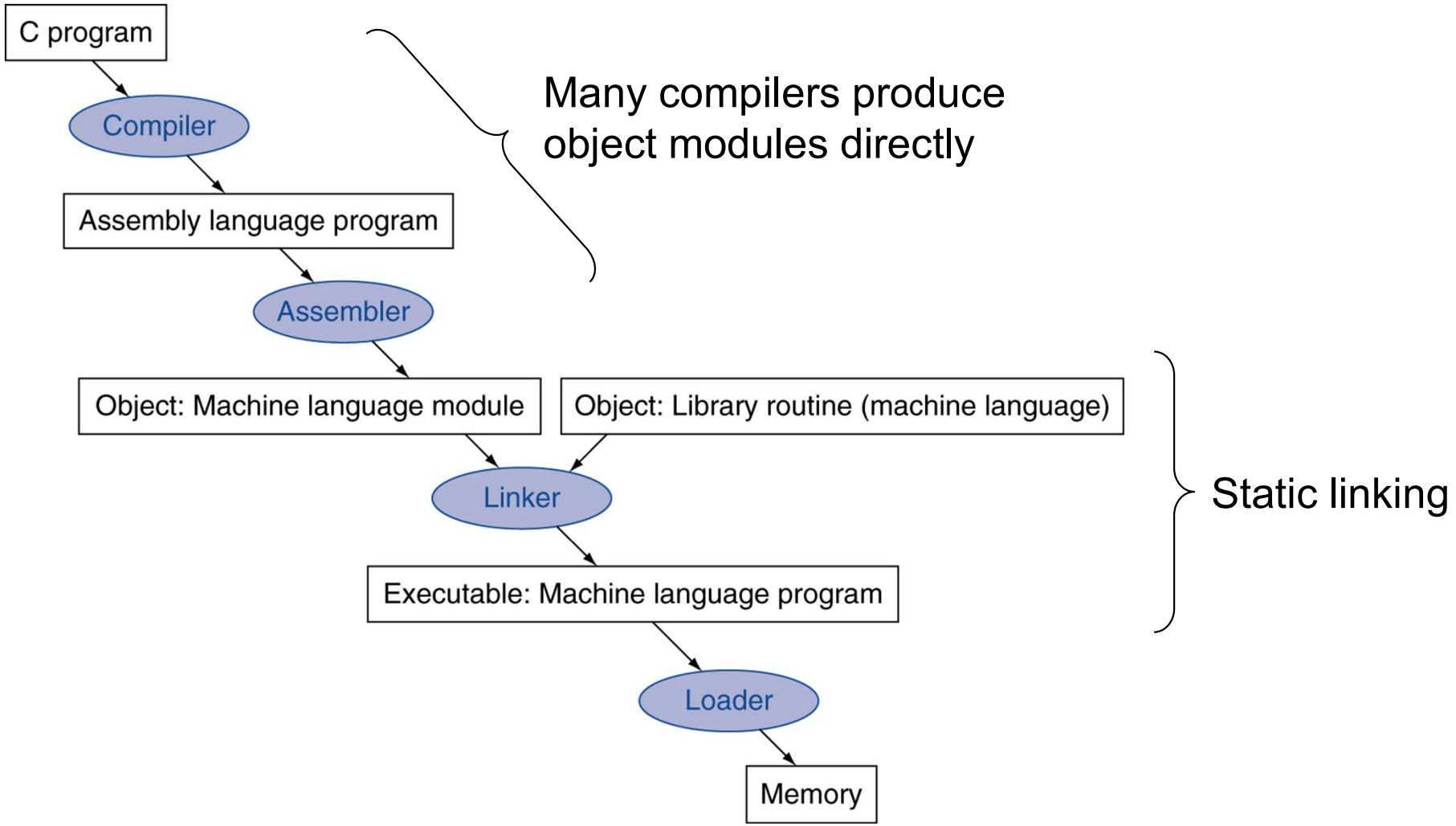
Addressing Mode Summary



LEGv8 Encoding Summary

Name	Fields							Comments
Field size		6 to 11 bits	5 to 10 bits	5 or 4 bits	2 bits	5 bits	5 bits	All LEGv8 instructions are 32 bits long
R-format	R	opcode	Rm	shamt		Rn	Rd	Arithmetic instruction format
I-format	I	opcode	immediate			Rn	Rd	Immediate format
D-format	D	opcode	address		op2	Rn	Rt	Data transfer format
B-format	B	opcode	address					Unconditional Branch format
CB-format	CB	opcode	address				Rt	Conditional Branch format
IW-format	IW	opcode	immediate				Rd	Wide Immediate format

Translation and Startup

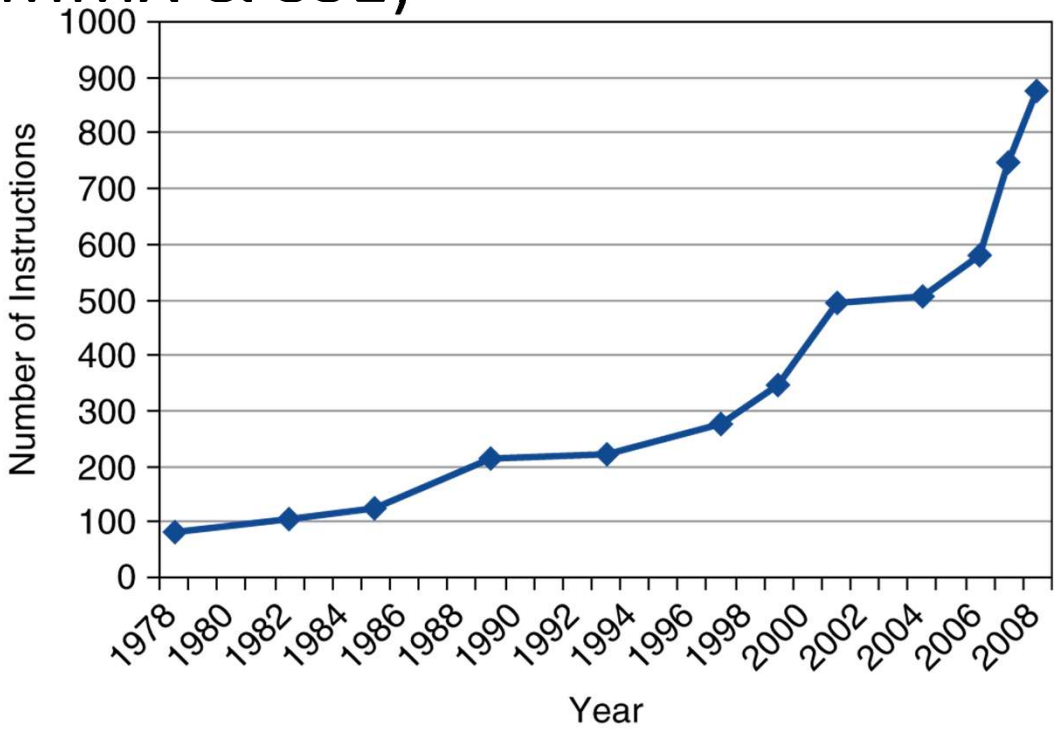


Fallacies

- Powerful instruction ⇒ higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code ⇒ more errors and less productivity

Fallacies

- Backward compatibility ⇒ instruction set doesn't change
  - But they do add more instructions (extensions like MMX & SSE)



x86 instruction set

Concluding Remarks

- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands good compromises
- Layers of software/hardware
  - Compiler, assembler, hardware
- LEGv8: typical of RISC ISAs
  - c.f. Intel x86 (CISC)